

Skill 7-oss-2520090235

SESSION 13

Task 1: To Parse Input Redirection, Open Files, Redirect stdin,
Handle Missing Files, Restore Streams, Test File Inputs

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 s71.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <fcntl.h>  
#include <sys/wait.h>  
#include <errno.h>  
  
#define MAX_LINE 1024  
#define MAX_ARGS 64  
  
int tokenize(char *line, char **args) {  
    int count = 0;  
    char *token = strtok(line, " \\t\\n");  
  
    while (token != NULL && count < MAX_ARGS - 1) {  
        args[count++] = token;  
        token = strtok(NULL, " \\t\\n");  
    }  
    args[count] = NULL;  
    return count;  
}  
  
char* parse_input_redirection(char **args) {  
    char *input_file = NULL;  
  
    for (int i = 0; args[i] != NULL; i++) {  
        if (strcmp(args[i], "<") == 0) {  
            if (args[i + 1] == NULL) {  
                fprintf(stderr, "Syntax Error: Expected filename after '<'\n");  
                return (char *)-1;  
            }  
  
            input_file = args[i + 1];  
  
            int j = i;  
            while (args[j + 2] != NULL) {  
                args[j] = args[j + 2];  
                j++;  
            }  
            args[j] = NULL;  
            break;  
        }  
    }  
    return input_file;  
}
```

[Read 114 lines]

```

int main(void) {
    char line[MAX_LINE];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            break;
        }

        int num_args = tokenize(line, args);
        if (num_args == 0) {
            continue;
        }

        if (strcmp(args[0], "exit") == 0) {
            break;
        }

        char *input_file = parse_input_redirection(args);
        if (input_file == (char *)-1) {
            continue;
        }

        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
            continue;
        }

        if (pid == 0) {
            // Child Process
            if (input_file != NULL) {
                // Open File
                int fd = open(input_file, O_RDONLY);

                // Handle Missing Files
                if (fd < 0) {
                    fprintf(stderr, "myshell: %s: %s\n", input_file, strerror(errno));
                    exit(EXIT_FAILURE);
                }
            }
        }
    }
}

```

```

    if (pid == 0) {
        // Child Process
        if (input_file != NULL) {
            // Open File
            int fd = open(input_file, O_RDONLY);

            // Handle Missing Files
            if (fd < 0) {
                fprintf(stderr, "myshell: %s: %s\n", input_file, strerror(errno));
                exit(EXIT_FAILURE);
            }

            // Redirect stdin
            if (dup2(fd, STDIN_FILENO) < 0) {
                perror("dup2 failed");
                close(fd);
                exit(EXIT_FAILURE);
            }

            close(fd);
        }

        execvp(args[0], args);
        fprintf(stderr, "myshell: command not found: %s\n", args[0]);
        exit(EXIT_FAILURE);
    } else {
        // Parent Process (Main shell restores standard stream control automatically)
        int status;
        waitpid(pid, &status, 0);
    }
}

return 0;

```

Output:

```
rohita@DESKTOP-1UHU58C:~$ gcc s71.c
rohita@DESKTOP-1UHU58C:~$ ./a.out
myshell> cat < shell.c
myshell: shell.c: No such file or directory
myshell> exit
rohita@DESKTOP-1UHU58C:~$
```

```
rohita@DESKTOP-1UHU58C:~$ nano s71.c
rohita@DESKTOP-1UHU58C:~$ ./a.out
myshell> cat < test.txt
myshell: test.txt: No such file or directory
myshell> line 1:
myshell: command not found: line
myshell> Apple
myshell: command not found: Apple
myshell> line 2: banana
myshell: command not found: line
myshell> grep "Banana" < test.txt
myshell: test.txt: No such file or directory
myshell> Line 2: Banana
myshell: command not found: Line
myshell> cat <
Syntax Error: Expected filename after '<'
myshell> exit
rohita@DESKTOP-1UHU58C:~$ s_
```

Observation: The dup2() call replaces standard input (STDIN_FILENO) with the opened file descriptor inside the child process, feeding file data directly into the executed command.

Because file table modifications are confined to the child process, the parent shell automatically retains keyboard input access when the child terminates.

Task 2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

Code:

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s72.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_LINE 1024
#define MAX_ARGS 64

int tokenize(char *line, char **args) {
    int count = 0;
    char *token = strtok(line, " \\t\\n");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }
    args[count] = NULL;
    return count;
}

char* parse_output_redirection(char **args, int *is_append) {
    char *output_file = NULL;
    *is_append = 0;

    for (int i = 0; args[i] != NULL; i++) {
        if (strcmp(args[i], ">") == 0 || strcmp(args[i], ">>") == 0) {
            if (strcmp(args[i], ">>") == 0) {
                *is_append = 1;
            }

            if (args[i + 1] == NULL) {
                fprintf(stderr, "Syntax Error: Expected filename after redirection symbol\\n");
                return (char *)-1;
            }

            output_file = args[i + 1];

            int j = i;
            while (args[j + 2] != NULL) {
                args[j] = args[j + 2];
                j++;
            }
            args[j] = NULL;
        }
    }
}
```

```

    }
    return output_file;
}

int main(void) {
    char line[MAX_LINE];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            break;
        }

        int num_args = tokenize(line, args);
        if (num_args == 0) continue;

        if (strcmp(args[0], "exit") == 0) break;

        int is_append = 0;
        char *output_file = parse_output_redirection(args, &is_append);
        if (output_file == (char *)-1) continue;

        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
            continue;
        }

        if (pid == 0) {
            if (output_file != NULL) {
                int flags = O_WRONLY | O_CREAT | (is_append ? O_APPEND : O_TRUNC);
                int fd = open(output_file, flags, 0644);

                if (fd < 0) {
                    fprintf(stderr, "myshell: %s: %s\n", output_file, strerror(errno));
                    exit(EXIT_FAILURE);
                }

                if (dup2(fd, STDOUT_FILENO) < 0) {
                    perror("dup2 failed");
                    close(fd);
                    exit(EXIT_FAILURE);
                }

                close(fd);

                execvp(args[0], args);
                fprintf(stderr, "myshell: command not found: %s\n", args[0]);
                exit(EXIT_FAILURE);
            } else {
                // Parent Process
                int status;
                waitpid(pid, &status, 0);
            }
        }

        return 0;
    }
}

```

Output:

```
myshell> exit
rohita@DESKTOP-1UHU58C:~$ nano 72.c
rohita@DESKTOP-1UHU58C:~$ nano s72.c
rohita@DESKTOP-1UHU58C:~$ gcc s72.c
rohita@DESKTOP-1UHU58C:~$ ./a.out
myshell> echo "Hello,World!" > output.txt
myshell> echo "Appended Line" >> output.txt
myshell> cat output.txt
"Hello,World!"
"Appended Line"
myshell> echo "Test" > /root/protected.txt
myshell: /root/protected.txt: Permission denied
myshell> exit
```

Observation: The program uses `dup2()` inside a child process to send standard output (stdout) to a file, either overwriting (`>`) or appending (`>>`) data. If a permission error occurs, the child prints an error message, leaving the main shell safely unaffected and working normally.

SESSION 14

Task 1: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

Code:

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s73.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_LINE 1024
#define MAX_ARGS 64

int tokenize(char *line, char **args) {
    int count = 0;
    char *token = strtok(line, " \\t\\n");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }
    args[count] = NULL;
    return count;
}

char* parse_append_operator(char **args) {
    char *output_file = NULL;

    for (int i = 0; args[i] != NULL; i++) {
        if (strcmp(args[i], ">>") == 0) {
            if (args[i + 1] == NULL) {
                fprintf(stderr, "Syntax Error: Expected filename after '>>'\n");
                return (char *)-1;
            }

            output_file = args[i + 1];

            int j = i;
            while (args[j + 2] != NULL) {
                args[j] = args[j + 2];
                j++;
            }
            args[j] = NULL;
            break;
        }
    }
    return output_file;
}
```

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s73.c *

int main(void) {
    char line[MAX_LINE];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            break;
        }

        int num_args = tokenize(line, args);
        if (num_args == 0) continue;

        if (strcmp(args[0], "exit") == 0) break;

        char *output_file = parse_append_operator(args);
        if (output_file == (char *)-1) continue;

        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
            continue;
        }

        if (pid == 0) {
            if (output_file != NULL) {
                int flags = O_WRONLY | O_CREAT | O_APPEND;
                int fd = open(output_file, flags, 0644);

                if (fd < 0) {
                    fprintf(stderr, "myshell: %s: %s\n", output_file, strerror(errno));
                    exit(EXIT_FAILURE);
                }

                if (dup2(fd, STDOUT_FILENO) < 0) {
                    perror("dup2 failed");
                    close(fd);
                    exit(EXIT_FAILURE);
                }

                close(fd);
            }

            execvp(args[0], args);
            fprintf(stderr, "myshell: command not found: %s\n", args[0]);
            exit(EXIT_FAILURE);
        } else {
            int status;
            waitpid(pid, &status, 0);
        }
    }

    return 0;
}
```

```
        close(fd);
    }

    execvp(args[0], args);
    fprintf(stderr, "myshell: command not found: %s\n", args[0]);
    exit(EXIT_FAILURE);
} else {
    int status;
    waitpid(pid, &status, 0);
}
}

return 0;
}
```

Output:

```
myshell> echo "Original Line" > log.txt
"Original Line" > log.txt
myshell> echo "Appended Line 1" >> log.txt
myshell> echo "Appended Line 2" >> log.txt
myshell> cat log.txt
Original Line
Appended Line 1
Appended Line 2
"Appended Line 1"
"Appended Line 2"
myshell> echo "Next Entry" >> /proc/invalid.txt
myshell: /proc/invalid.txt: No such file or directory
myshell> exit
```

Observation: Using O_APPEND with open() attaches new output to the end of the file, preserving all existing text without deleting anything. dup2() redirects the output stream to the file inside the child process, leaving the main shell terminal running cleanly as usual.

Task 2: To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 s74.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <fcntl.h>  
#include <sys/wait.h>  
#include <errno.h>  
  
#define MAX_LINE 1024  
#define MAX_ARGS 64  
  
int tokenize(char *line, char **args) {  
    int count = 0;  
    char *token = strtok(line, " \\t\\n");  
    while (token != NULL && count < MAX_ARGS - 1) {  
        args[count++] = token;  
        token = strtok(NULL, " \\t\\n");  
    }  
    args[count] = NULL;  
    return count;  
}  
  
char* parse_stderr_redirection(char **args) {  
    char *error_file = NULL;  
  
    for (int i = 0; args[i] != NULL; i++) {  
        if (strcmp(args[i], ">") == 0) {  
            if (args[i + 1] == NULL) {  
                fprintf(stderr, "Syntax Error: Expected filename after '>'\n");  
                return (char *)-1;  
            }  
  
            error_file = args[i + 1];  
  
            int j = i;  
            while (args[j + 2] != NULL) {  
                args[j] = args[j + 2];  
                j++;  
            }  
            args[j] = NULL;  
            break;  
        }  
    }  
    return error_file;  
}
```

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s74.c *

int main(void) {
    char line[MAX_LINE];
    char *args[MAX_ARGS];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            break;
        }

        int num_args = tokenize(line, args);
        if (num_args == 0) continue;

        if (strcmp(args[0], "exit") == 0) break;

        char *error_file = parse_stderr_redirection(args);
        if (error_file == (char *)-1) continue;

        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
            continue;
        }

        if (pid == 0) {
            if (error_file != NULL) {
                int fd = open(error_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

                if (fd < 0) {
                    fprintf(stderr, "myshell: %s: %s\n", error_file, strerror(errno));
                    exit(EXIT_FAILURE);
                }

                if (dup2(fd, STDERR_FILENO) < 0) {
                    perror("dup2 failed");
                    close(fd);
                    exit(EXIT_FAILURE);
                }

                close(fd);
            }
        }
    }
}

```

```

        execvp(args[0], args);
        fprintf(stderr, "myshell: command not found: %s\n", args[0]);
        exit(EXIT_FAILURE);
    } else {
        int status;
        waitpid(pid, &status, 0);
    }
}

return 0;

```

Output:

```

rohita@DESKTOP-1UHU58C:~$ ./a.out
myshell> ls non_existent_file.txt 2> error.log
myshell> cat error.log
ls: cannot access 'non_existent_file.txt': No such file or directory
myshell> ls 2> /root/protected.log
myshell: /root/protected.log: Permission denied
myshell> exit
rohita@DESKTOP-1UHU58C:~$

```

Observation:

`dup2()` redirects **stderr (2)** to a file in the child process. So, error messages go to the file instead of the terminal. The **parent shell is not affected** and continues normally.